

How to Handle Multi-Class Output

- ❑ This is an issue with any learning model which only supports binary classification (perceptron, SVM, etc.)
- ❑ Create 1 perceptron for each output class, where the training set considers all other classes to be negative examples
 - ❑ Run all perceptrons on novel data and set the output to the class of the perceptron which outputs high. If there is a tie, choose the perceptron with the highest net value
- ❑ Create 1 perceptron for each pair of output classes, where the training set only contains examples from the 2 classes
 - ❑ Run all perceptrons on novel data and set the output to be the class with the most wins (votes) from the perceptrons. In case of a tie, use the net values to decide
 - ❑ Number of models grows by the square of the output classes

Objective Functions: Accuracy/Error

- ❑ How do we judge the quality of a particular model (e.g. Perceptron with a particular setting of weights)
- ❑ Consider how accurate the model is on the data set
 - ❑ *Classification accuracy* = # Correct/Total instances
 - ❑ *Classification error* = # Misclassified/Total instances (= $1 - \text{acc}$)

How to achieve the objective?

- ❑ Usually minimize a Loss function (aka cost, error)
- ❑ For real valued outputs and/or targets
 - ❑ Pattern error = Target – output
 - ❑ Errors could cancel each other: $\sum |t_i - z_i|$ (L1 loss)
 - ❑ Common approach is *Squared Error* = $\sum (t_i - z_i)^2$ (L2 loss)
 - ❑ Total sum squared error = $\sum$ Pattern Errors
$$= \sum (t_i - z_i)^2$$
- ❑ For nominal data, pattern error is typically 1 for a mismatch and 0 for a match
 - ❑ For nominal (including binary) output and targets, SSE and classification error are equivalent